

Säkerhet i AI-system (DV2607)

[Rapport för inlämningsuppgift]

Namn student 1: Rasmus Johansson

E-post student 1: rajo22@student.bth.se

Namn student 2: Rasmus Dunder

E-post student 2: radn21@student.bth.se

Part 1: Centrala adversarial input attacker

Del 1.1) Beskrivning av adversarial input attacker

Adversarial input attacker handlar i grunden om att man medvetet lägger till små, noggrant beräknade störningar i en bild eller annan typ av input för att lura en maskininlärningsmodell att klassificera fel. Det som gör dessa attacker så kraftfulla är att störningarna ofta är helt osynliga för oss människor, men ändå påverkar modellen starkt eftersom de är skapade utifrån modellens egna svagheter. Syftet med en adversarial attack är alltså att utnyttja hur modellen faktiskt fungerar, och vrida inputen precis så mycket att modellen tappar förmågan att se det verkliga innehållet.

I vårt projekt valde vi att gå vidare med just **Targeted PGD** som specifik attackmetod. Anledningen till det är att Targeted PGD inte bara försöker få modellen att göra ett slumpmässigt misstag, utan systematiskt styr modellen mot en helt bestämd felaktig klass genom iterativa, små och kontrollerade perturbationer. Genom att uppdatera störningen stegvis, samtidigt som den hålls inom ett begränsat område, så blir attacken betydligt mer kraftfull och stabil än enklare metoder.

Detta gör Targeted PGD särskilt användbart i vårt projekt eftersom vi både kan styra modellen mot en förutbestämd felaktig klass och samtidigt analysera hur modellen successivt förlorar sin robusthet. Den iterativa karaktären ger oss en mer detaljerad och kontrollerad förståelse av hur små förändringar påverkar klassificeringen, och gör det lättare att reproducera och utvärdera attackens effektivitet.

Del 1.2) Implementation av adversarial input attack

En PGD-attack fungerar genom att man lägger till små, iterativa och avsiktliga störningar i input-datan för att få modellen att misstolka vad den faktiskt ser. Till skillnad från den enklare FGSM-metoden beräknas gradienten av loss-funktionen upprepade gånger, och perturbationen uppdateras steg för steg samtidigt som den projiceras tillbaka inom ett fördefinierat ϵ -område. På så sätt kan attacken mer effektivt maximera loss och successivt driva klassificeringen bort från det korrekta svaret.

Vi valde att arbeta med en **Targeted PGD**. Skillnaden mellan en vanlig (untargeted) PGD-attack och en targeted version är att man inte bara försöker få modellen att ha fel, utan att man aktivt styr den mot en specifik felaktig klass. I stället för att enbart maximera loss gentemot den korrekta klassificeringen optimerar man alltså modellen mot just den klass man vill att inputen ska misstas för. Detta gör attacken betydligt mer kontrollerad men också mer beräkningsmässigt krävande jämfört med en enkel FGSM-variant.

För att implementera detta följde vi PyImageSearch-guiden för targeted adversarial attacks (länk finns i noteboken). Vi använde därför modulerna TensorFlow, NumPy och OpenCV (cv2), vilket gav oss möjligheten att både manipulera bilderna och snabbt beräkna de gradienter och upprepade uppdateringar som krävs för PGD-attacken. Vi experimenterade även med olika steps-värden för att hitta en störningsnivå som var tillräckligt

subtil för att inte synas men ändå effektiv nog att lura modellen konsekvent mot den specifika målklassen.

Del 1.3) Säkerhetsåtgärder

Vi tycker att den bästa säkerhetsåtgärden mot vår Targeted PGD-attack borde vara att implementera **Adversarial Training**. Adversarial training fungerar genom att man genererar manipulerade bilder med hjälp av PGD och Targeted PGD, och sedan tränar modellen på dessa attackerade exempel. På så sätt "vet" modellen att sådana störningar kan förekomma och lär sig att inte bli lurad av dem i framtiden. Till exempel: om bilder som egentligen föreställer en panda har manipulerats för att likna en traktor, och modellen tränas på både originalet och den manipulerade versionen, så blir den mer robust mot just den typen av angrepp.

Adversarial training använder själva attackmetoden som grund för försvaret och därmed tränar modellen på exakt den typ av manipulation den kommer utsättas för.

Vi valde dock att inte implementera adversarial training skydder framför andra metoder, trots vi anser att det ger det mest direkta och kraftfulla skyddet. Dettas eftersom implementationen av detta skydd är mer komplext. Istället så tittade vi på försvar som *randomized smoothing*, *confidence-based defenses* och *label smoothing*.

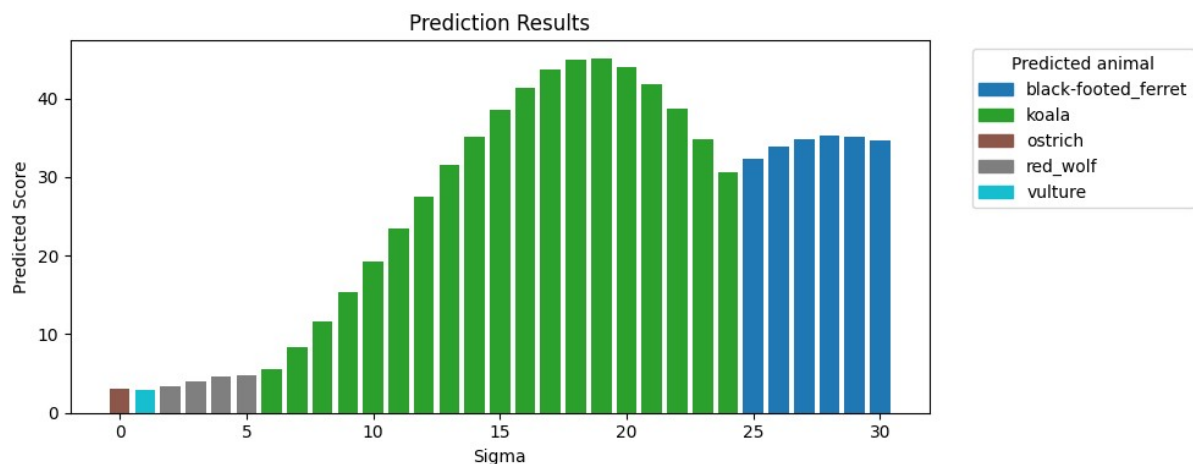
Enligt oss är alla dessa svagare i just vårt scenario, eftersom de inte blockerar lika många attacker som adversarial training gör. Man kan argumentera för att vissa av de andra försvarerna ger ett mer generellt skydd, även om det, just i vår uppgift, målet är att försvara sig mot den attack vi själva byggt och det är mer effektivt att ha ett försvar som specifikt är inriktat på just de bilder och störningar som skapas av vår attackmetod.

Del 1.4) Implementering av skydd (frivillig för betyg A eller B)

Det skydd som vi valde att implementera är *randomized smoothing*.

Randomized smoothing fungerar genom att lägga till slumpässigt Gaussiskt-brus och klassifiera på nytt, detta upprepas **N** gånger och snitt klassifiering skickas tillbaka. Idéen är att Gaussiskt brus är större än adversiella pertuberingar och så sett påverkar det modellen inte lika hårt.

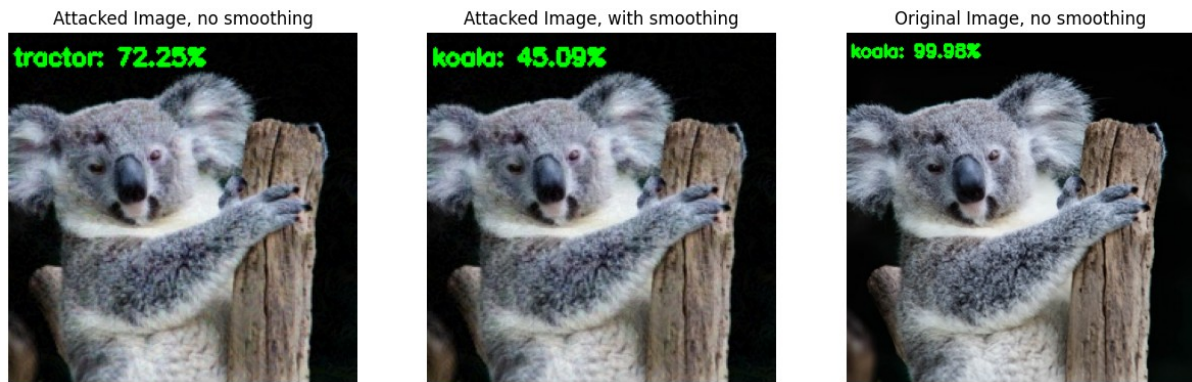
För att se till att modellen presterar så bra som möjligt gjordes några tester där vi alternerade på sigma värdet till funktionen för att se vilket som presterade bäst, sedan plottades resultatet för analys.



Figur 1: Resultat från test

Från ploten kan vi se hur olika sigma värden påverkar modellen där sigma värden mellan 6 och 24 gör rätt prediktion på bilden, starkast är vid sigma = 19.

Modellen kördes på nytt både på den attackerade bilden och ursprungsbilden för att se hur väl skyddet presterar. Originalbilden, efter randomized smoothing, fick en 99.98% confidence score att vara en koala, medan den attackerade bilden fick en 45.09% sannolikhet att vara en koala.



Figur 2: Klassifieringar på bilder

Vi ser detta som ett fungerade skydd, dock svagt, för Targeted PGD attacker.

Att som tidigare nämnt implementera en Adversarial Training model hade potentiellt varit ett bättre skydd.

Part 2: Federated learning scenario

Part 2.1)

We implemented Flower's PyTorch example (see:

<https://flower.ai/docs/framework/tutorial-quickstart-pytorch.html>) and added a adversarial perturbation attack to use for the tests.

When comparing the different federated learning scenarios, we can clearly see a decrease in accuracy when adding malicious clients. For the IID models, the accuracy drops from the baseline of 0.6 to 0.4 with one malicious client, and further down to 0.2 with two malicious clients.

We observe a similar trend in the non-IID models, although the baseline accuracy is slightly lower compared to the IID case. This lower starting accuracy continues to decline as we add malicious clients, showing that the attack negatively impacts performance here as well.

Initially, we expected the non-IID models to struggle more with the attack, since each client has different data and the global model might be less able to detect inconsistencies in client updates. Interestingly, we now observe that the non-IID models handle the attack much better. Although the non-IID models start with slightly lower baseline accuracy, the relative impact of malicious clients is smaller compared to the IID case. This indicates that the non-IID data distribution provides some robustness against the attack, reducing its overall effect.

One possible explanation is that in non-IID settings, the global model is already influenced more strongly by certain clients with larger or more dominant data partitions. This means that while individual malicious updates might stand out less compared to IID, their impact is somewhat "diluted" across the uneven data, resulting in a performance drop that is noticeable but not drastically worse than in the IID scenario. In other words, non-IID models are vulnerable, but the distribution of the data can partially buffer or amplify the effect depending on which clients are attacked.

Overall, these results demonstrate that federated learning systems are vulnerable to malicious clients, regardless of whether the data is IID or non-IID. The data portioning model don't matter that much.

Part 2.2)

When comparing FedProx and FedAvg, we observed that overall model performance was similar across most scenarios. For the IID models, FedAvg handles the attack slightly better than FedProx, maintaining higher accuracy and slightly improved F1, Kappa, and ROC values. This is expected, since FedAVG is well-suited for IID data, where all clients share similar distributions; in contrast, FedProx introduces a proximal term to mitigate heterogeneity,

which is unnecessary in IID settings.

For the non-IID models, performance trends remained largely similar between FedAvg and FedProx, with baseline accuracy slightly lower than IID and decreasing in the presence of malicious clients, consistent with patterns seen under both algorithms. These results indicate that, for this specific experiment, FedProx does not provide a significant advantage over FedAvg in mitigating the impact of malicious clients or improving overall model performance. Both aggregation strategies are affected similarly by the perturbation attack, and adding one or more attackers continues to degrade model performance in comparable ways.

Overall, this suggests that while FedProx is designed to handle heterogeneity in client updates, in our setup with 5 clients and the chosen attack, its impact is minimal. The system remains vulnerable to malicious clients, regardless of whether FedAvg or FedProx is used as the aggregation algorithm, and FedAvg is slightly more effective on IID data due to the homogeneity of client updates.

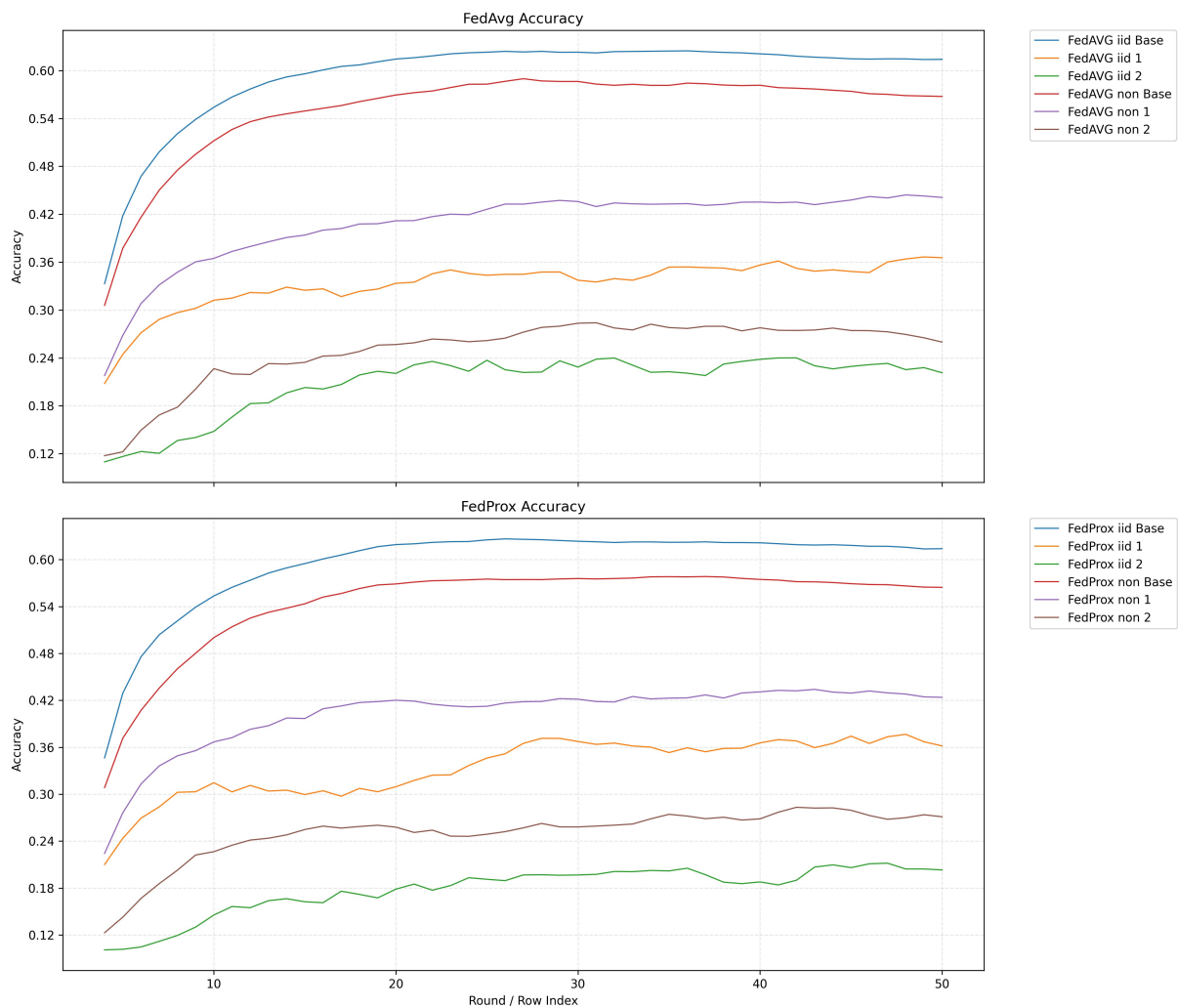
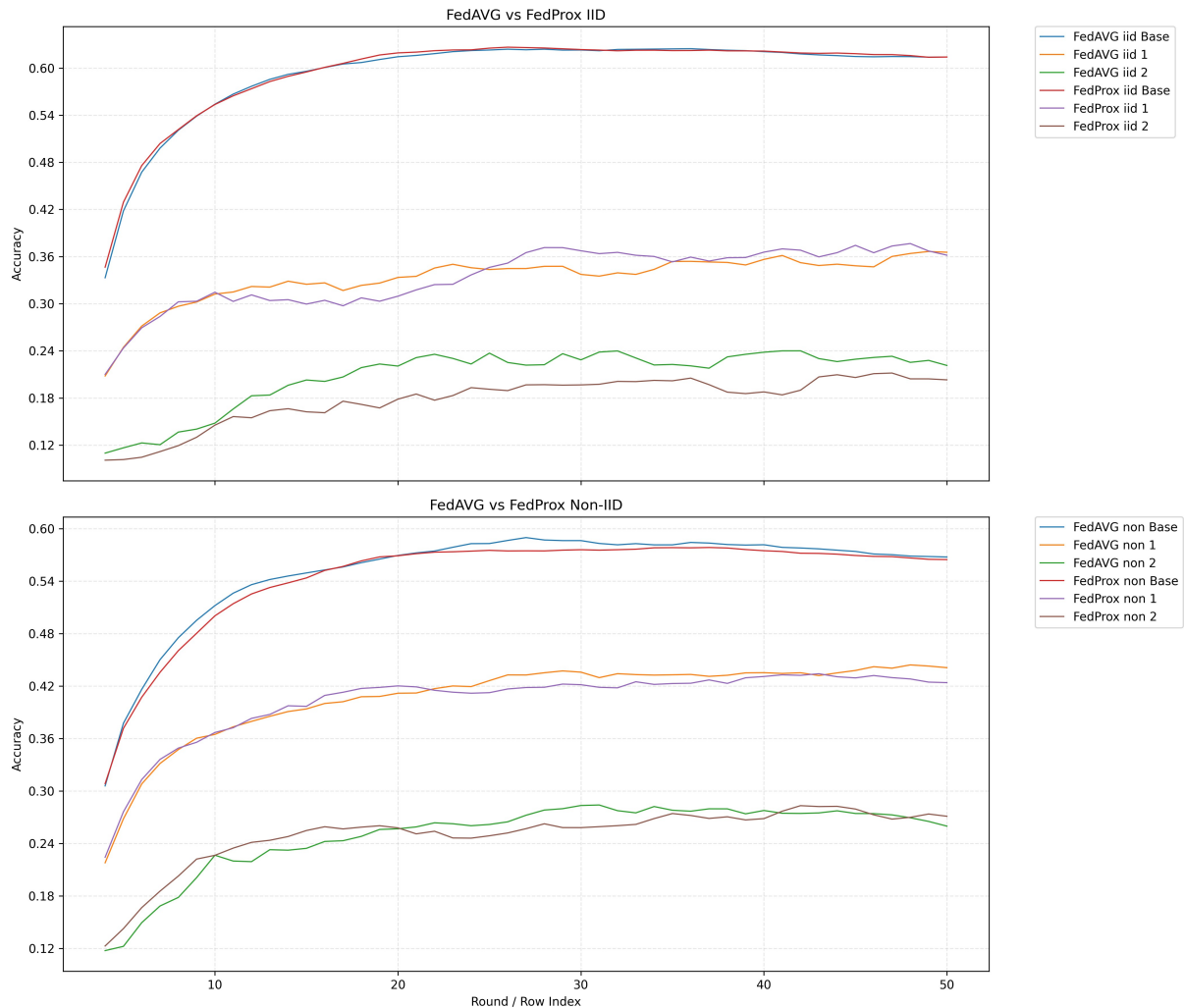


Figure 3: Accuracy plots of tests



Figur 4: Accuracy plots of tests

Del 2.3) Implemenation of protection ([option if you aim for A and/or B grade](#))

For the first defense we decided to use the Federated Trimmed Average strategy. This defense works almost like the usual Federated Average, but instead of averaging all client updates, it removes the lowest and highest 0.1 fractions before averaging. By trimming away these extreme values, the server often excludes the attacked or heavily manipulated client updates.

When looking at our results we can clearly see that Trimmed Average handles the attacks much better. With one malicious client, the accuracy barely drops at all. With two malicious clients, the Trimmed Average strategy still performs noticeably better than the normal averaging strategy, which already struggles even with a single malicious client.

The drop in accuracy at two malicious clients can be explained by the fraction of removed updates. Since our environment only has 5 clients, having 2 malicious ones means that 40% of the clients are adversarial, which is already close to the threshold that a 0.1 trimming fraction can tolerate. Increasing the trimming fraction might help improve accuracy in this scenario because more of the poisoned updates would be removed. However, increasing this parameter also comes with a downside: trimming away too many updates can reduce the contribution of the honest clients and therefore lower the overall accuracy in the baseline (non-attack) setting or when only a small number of clients participate.

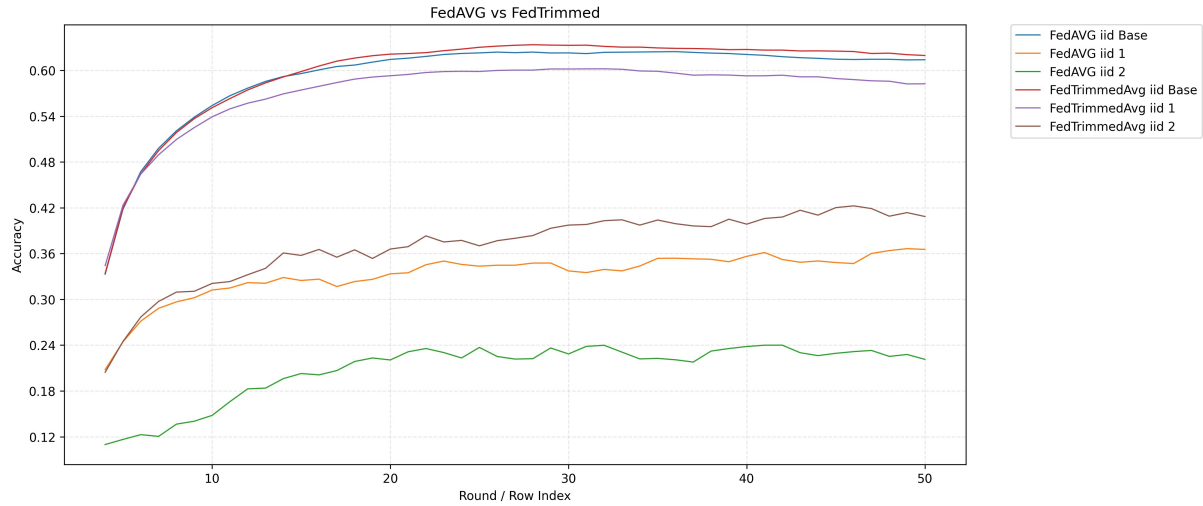


Figure 5: Accuracy plots of tests

Del 2.4) Implementation of protection (option if you aim for A)

The second defense we decided to use was the Federated Median strategy. This defense also works similarly to Federated Average, but instead of taking the average of the client updates, it takes the coordinate-wise median. Because the median is naturally robust to extreme values, any malicious updates with unusually large or targeted manipulations have much less influence on the final aggregated model.

From our results, we can clearly see that the median defense performs much better than the Federated Trimmed Average when there are two malicious clients. This difference is likely due to the same fractioning limitation we observed with Trimmed Average. Since Trimmed Average removes only a fixed fraction of updates (0.1 from each side), it can fail when the number of malicious clients exceeds what the trimming fraction can handle.

We also see that both strategies perform equally well in the Base case and with only one malicious client, which makes sense because both defenses are strong enough to filter out a single outlier. However, the Median strategy performs better as the number of malicious clients increases. The reason is that the median always selects the central update regardless of how many extreme (and potentially malicious) updates exist. As long as the majority of clients are honest, the median will still land on an honest update, meaning additional malicious clients do not affect it as strongly.

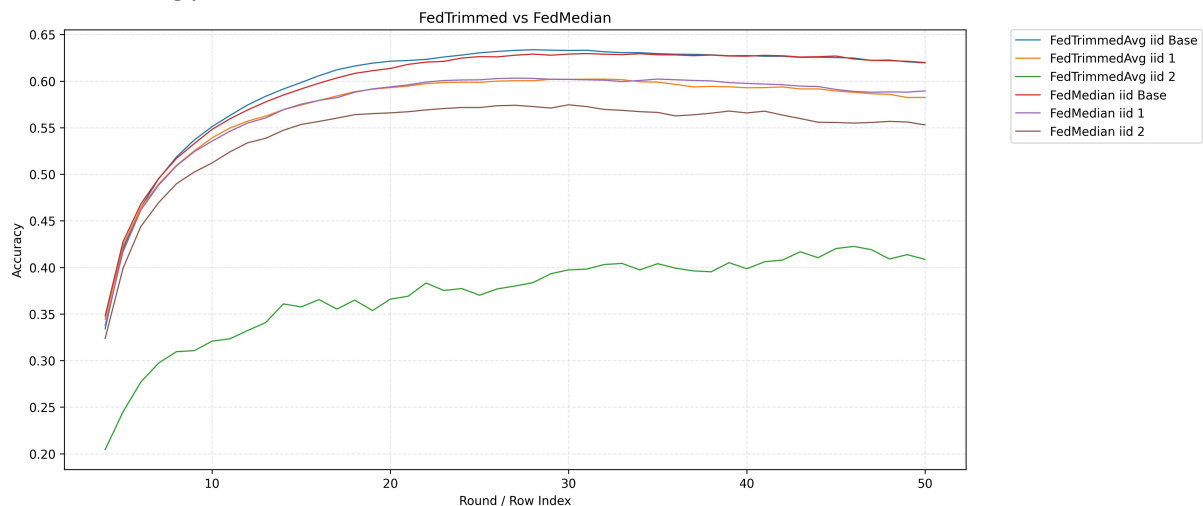


Figure 6: Accuracy plots of tests